What we learned about MCP by building brightplace Connect

Takeaways from building an MCP for the future of rental search

Renters are now starting their apartment searches with AI assistants. They open Claude or ChatGPT, describe what they want, and expect it to do the legwork. We wanted brightplace reachable at that moment, so we built **brightplace Connect**, a Model Context Protocol (MCP) server that lets those assistants search listings, pull details, check tour availability, and book a tour on a renter's behalf.

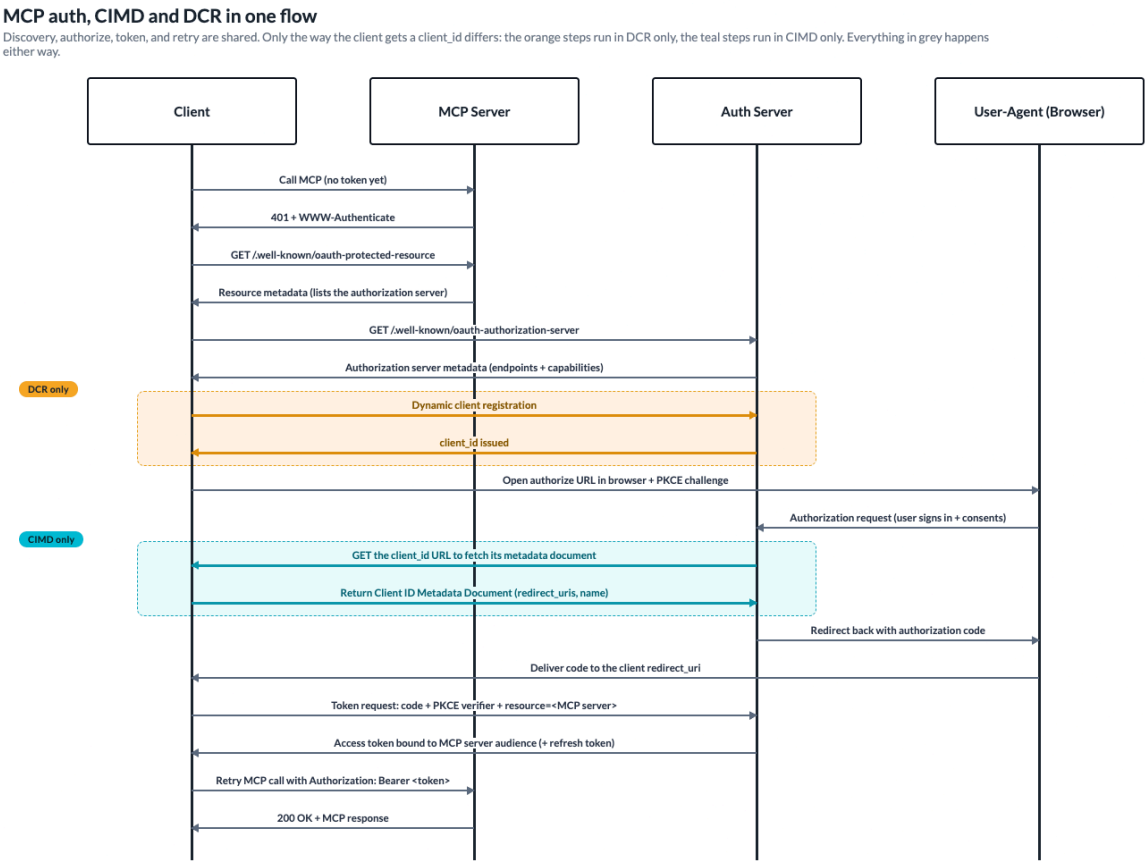
Going in, I figured the hard part would be the features, tools, data, and logic. It wasn't; The hardest part was getting clients to connect, and the real lesson of the whole project is that the real difference between an API and an MCP is authentication.

Here's what we learned, along with the engineering that supports it.

Authentication isn't automatic

The mechanics of the MCP handshake are well-specified; the trouble is that the platforms each implement a different slice of a spec that's still moving.

How authentication works



Under the MCP authorization spec, authentication is OAuth 2.1, and your MCP server is the resource server, not the authorization server. That separation is something you need to design for.

The handshake is a fixed sequence. A client calls your server without a token; you return a 401 with a WWW-Authenticate header pointing to your metadata. The client fetches your Protected Resource Metadata at /.well-known/oauth-protected-resource (RFC 9728) to learn which authorization server to trust, then fetches that server's metadata at /.well-known/oauth-authorization-server (RFC 8414) to find the authorize and token endpoints. From there, it's the authorization-code flow with PKCE (RFC 7636, S256 only): the client generates a code_verifier, opens the authorize URL in the browser, the user consents, and the returned code is exchanged for an access token using the verifier. That token is bound to your server as its audience via a resource indicator (RFC 8707), so you reject anything minted for another service and never forward a client's token upstream. Every subsequent call includes the Authorization: Bearer <token> header.

The one step with real variation is how the client obtains a client_id. With **CIMD**, the client publishes a small JSON metadata document at an HTTPS URL, and that URL *serves as* its client_id; your authorization server dereferences and validates the document at authorization time, with no registration round-trip. With **DCR** (RFC 7591), the client POSTs to a registration endpoint and your server mints a client_id on the spot, which both sides then have to persist. The November 2025 revision made CIMD the recommended default and pushed DCR down to a backward-compatibility fallback.

The issues we faced

None of those mechanisms is what bit us. PKCE is mandatory and well-specified, the discovery endpoints are fixed, and a bearer token is a bearer token. The trouble is that each platform picked a different registration path, built against a different revision of the spec:

- **Claude** uses Client ID Metadata Documents (CIMD).
- **ChatGPT** uses Dynamic Client Registration (DCR), but only with specific scopes.
- **Perplexity** also uses DCR, but drops the connection on us.

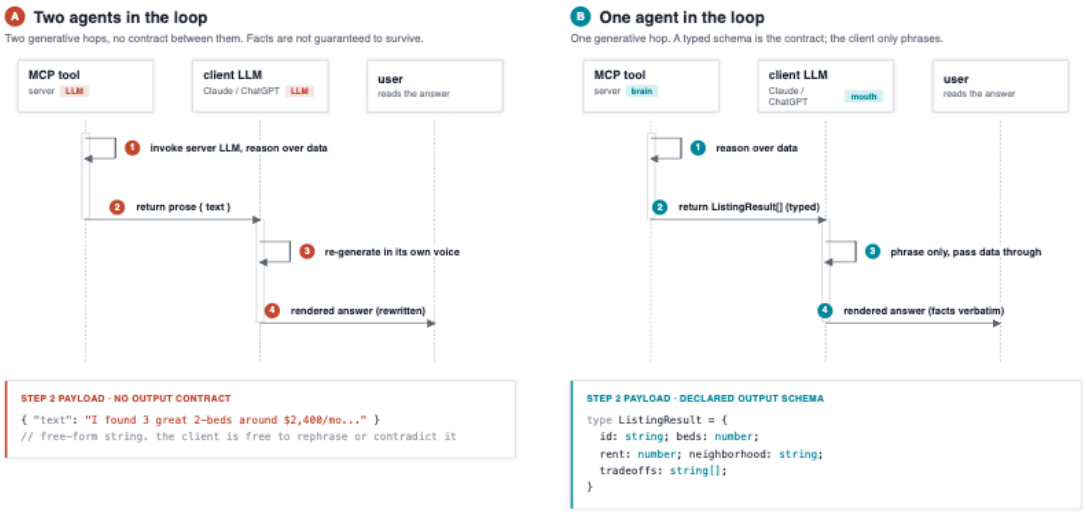
Three platforms, three auth models, three completely different ways to fail. The spec moved three times in 2025: OAuth 2.1 in March, the resource-server split and Protected Resource Metadata in June, and CIMD as the default in November, so the platforms are effectively implemented against different revisions of the same document. Claude tracks the newest and speaks CIMD; ChatGPT and Perplexity still take the DCR path. Supporting all three means implementing *both* registration models and keeping both healthy.

That's where the failures live. ChatGPT won't proceed unless the scopes it wants are advertised. Perplexity completes the handshake and then drops the session, a token-and-registration *lifecycle* problem, not a one-time connect problem, and the kind of thing you can only debug from inside your own auth server. We spent more time getting these handshakes to work than we did building the server's actual functionality.

The two-agent problem

You may imagine building your MCP server with an LLM on your backend to handle requests. If this is the case, your design now has *two* models in the loop, yours and the client's, and only one of them should be talking. We initially considered returning the natural language from our model, but quickly realized that would be a mistake.

If a tool returns natural language, you cannot guarantee that the language reaches the user. The calling LLM rewrites everything in its own voice. Best case, it's lossy, and your specifics get flattened. Worst case, the client disagrees with your output and contradicts it, leaving the user to watch two AIs argue. That kills trust fast.



The fix is a hard contract boundary: the server returns structured, typed data against a declared output schema, and the client is left to do the one thing it's genuinely good at, phrasing.

```json
{
 "listing_id": "bp_48213",
 "rent_monthly": 2400,
 "beds": 2,
 "available_on": "2026-07-15",
 "tradeoffs":  [...]
}
```

The rent, the IDs, and the tradeoffs survive because they're data, not natural language. And the schema does double duty: it's the contract that protects your facts, and, together with the tool description, it's the interface the client uses to decide *which* tool to call. The client picks tools almost entirely based on their descriptions, so we write those with the same care as a public function signature.

The MCP is the adapter, not the product

I've been pretty critical of "thin wrapper" products, apps that sit on top of a language model with no proprietary data or real capability underneath. If you can build it in an afternoon, so can everyone else.

But a thin wrapper over something real is a different story, and that's exactly what an MCP should be. We already had the Advisor, the data, and the reasoning pipeline, so brightplace Connect became an integration layer between what we'd built and the client calling it. The server is a small set of tools, each a thin adapter over an endpoint we already had. None of those are new capabilities; they're the existing surface, re-expressed as tools an agent can call.

That's the right way to think about it. The MCP is the adapter, not the product. The value isn't the MCP; it’s the output of the backend logic that the MCP then exposes. If there's nothing proprietary underneath, an MCP is worth nothing. If there is, it's the thinnest, fastest part of the whole build.

How it's wired

A few decisions in the layer between the MCP server and our own systems mattered more than anything in the spec.

The server is a trusted backend caller, not a token forwarder. It validates the inbound user token (audience check: was this issued for us?) and then calls the Advisor and internal services with its own, short-lived credentials. It never forwards the user's token upstream; the spec forbids it, and collapsing those trust boundaries is how one leaked token becomes a much bigger problem. The user's identity rides along as verified claims, not as something a client could spoof.

Latency budgets are per-tool, not per-server. The tools aren't doing comparable work, so a single target is the wrong model. The lookups, get_listing, and get_tour_availability, are record reads and should return fast. search_listings runs the full extraction-and-ranking pipeline with an LLM in the loop, so it gets a looser budget. Conflate the two, and you either make your reads look slow or set a bar your reasoning tools can't hit.

Compliance cannot be delegated across a trust boundary. Whatever policy governs your product has to apply identically through the MCP, because the client will relay whatever you return, and you don't control it. For us, that's Fair Housing: input filtering for protected-class steering, and the same response checks that govern the consumer experience, running on the server for every response. The MCP feels like plumbing, but it's a production surface talking to real users, so treat it like one.

What I’d tell anyone building an MCP

Roll your own auth server. This goes against the usual advice, and I mean it. Given how much time we burned fiddling with a third-party auth server and hitting dead ends on capabilities it just couldn't handle, we could have stood up our own in the same amount of time. Be clear about what you're signing up for: spec-compliant means implementing Protected Resource Metadata (RFC 9728), Authorization Server Metadata (RFC 8414), PKCE with S256, *both* CIMD and the DCR fallback, token rotation and refresh, and audience validation, and keeping pace with a spec that was revised three times last year. The payoff is full control and, more importantly, visibility into the internals when a connection can't be established. Since connection problems were the single biggest time sink of the project, that visibility is worth a lot. When Perplexity drops the session, you want to see exactly which step failed, not file a ticket against a black box.

MCP isn't a silver bullet. Everyone wants one right now, which is exactly when you should slow down and ask whether you actually need it. A few honest questions first:

- Can you get by with just an API? If you're exposing read-only data to one known client, a bearer-token-protected endpoint is far less work than a full OAuth 2.1 authorization server. The whole auth ordeal exists to solve the *many clients, many users, no human in the loop* problem. If you don't have that problem, don't buy that complexity.
- Is your MCP just a data feed, or does it have a reasoning layer? If it's the latter, the two-agent problem is yours to design around from day one.
- How will your output collide with the client agent on the other end? Decide where the brain ends, and the mouth begins, before you write a single tool.

Build an MCP because your use case calls for one, not because it's the next hot thing.

---

brightplace Connect is live at mcp.brightplace.ai/mcp. Docs at docs.brightplace.ai.

Thanks for reading.